Automated Ontology Development Using a Memory-Based MCP Approach

1. Introduction

This project develops an AI assistant tool to generate a domain-specific ontology by automating the Stanford Ontology Development 101[1] process using a memory-based MCP approach. The domain ontology specific development includes seven steps: (i) determining domain/range and competency question generation, (ii) reusing existing ontologies, (iii) enumerating important terms (concept extraction), (iv) definition of classes and their hierarchies (concept extraction), (v) properties for classes (concept extraction), (vi) domain and range definition, and constraint, and (vii) lastly creating instances. We design a Memory-based Model Context Protocol[2] (MCP) that automatically constructs a draft ontology and then validates it using a parser and reasoner at the end of construction. This memory-based MCP follows the steps above. The next section will describe our approaches in detail.

2. Memory-based Model Context Protocol

Memory-based MCP includes four main components: client, host, server, and memory.

1. **Client:** This component serves as an interface between the user and the host system, enabling the execution of goals defined within the ontology development process. The client submits these goals to the LLM Planner residing on the host, which orchestrates the necessary tasks to accomplish them. Upon completion, the system generates and returns a report detailing the results and outcomes of the executed goals. The goals are as follows: **memory generation** (including step ii), competency question generation (step i), ontology generation (steps iv to vii), mapping concepts with existing concepts (part of step ii), borrowing existing classes from mapping to the generated ontology, and validations and consistency. Memory generation is a prerequisite for all ontology-development tasks, as it provides access to both procedural knowledge (EU AI Act articles) and declarative knowledge (ontology development prompts and reused ontologies). These goals are converted into an execution plan by the LLM Planner in the Host and subsequently executed by the MCP client.
2. **Host:** This component converts goals into actions, creates a plan for those actions with LLM Planner, and returns this action to the Client. An MCP client triggers the Server to execute the action when the client calls it. Furthermore, it includes two agents: a document crawler and a GPT-based domain-expert agent.
3. **Server:** All these actions are executed by Ontology Generation in the Server. It has a connection with Agents in Host. The domain expert agent is called to perform steps triggered by the Ontology Generation.
4. **Memory:** Two different memory types, procedural and declarative, are used in this component. Procedural memory contains articles from the EU AI Act 2024, which are crawled by the document-crawler agent, whereas the declarative memory includes ontology development steps and rules in prompts and in existing ontologies. The memory generator binds these declarative and procedural memories into a pipeline for access on the server. Ontology Memory parses the ontologies to retrieve classes and properties in the server.

The proposed approach was used to generate an initial draft ontology structure, including candidate classes, hierarchies, and relationships. The resulting ontology was subsequently refined, validated, and completed through manual ontology engineering to improve conceptual consistency, align with the EU AI Act, and incorporate annotations and ontology design decisions.

---

[1] https://protege.stanford.edu/publications/ontology_development/ontology101.pdf, accessed on June 15, 2026.
[2] https://github.com/sefeoglu/eu-ai-act-ontology, accessed on June 15, 2026.

3. Evaluation

3.1 Data and Settings:

Existing Ontologies: AIRO[3], AIO[4], DPV[5], and VAIR[6]. *Documents*: The English version of the EU AI Act 2024[7]. *LLM:* Test with GPT-3.5-Pro and Production with GPT-5.4-mini. The other settings are available in our source code[8].

3.2 Competency Questions and SPARQLs

The following competency questions are evaluated on the ontology. The results are in the repository.

a.) Which high-risk AI systems are represented in the ontology?

```
SELECT ?system ?label
WHERE {
 ?system a euai:high_risk_ai_system .
 OPTIONAL {
  ?system rdfs:label ?label .
 }
}
ORDER BY ?label
```

b.) Which compliance-related activities, documentation requirements, and risk-management measures are associated with AI providers?

```
SELECT ?s ?p ?o
WHERE {
 ?s ?p ?o .
 FILTER(
  CONTAINS(STR(?p), "compliance") ||
  CONTAINS(STR(?p), "documentation") ||
  CONTAINS(STR(?p), "risk") ||
  CONTAINS(STR(?p), "oversight")
 )
}
ORDER BY ?s ?p
```

c.) Which transparency obligations are represented in the ontology, and do they require disclosure?

```
SELECT DISTINCT ?actor ?label
WHERE {
 ?actor a euai:Actor .
 ?obligation a euai:TransparencyObligation ;
        euai:appliesToActor ?actor .
 OPTIONAL {
  ?actor rdfs:label ?label .
 }
}
ORDER BY ?label
```

3.3 Validation and Metrics

Protégé and Reasoner: The ontology was loaded with rdflib. We also executed the HERMIT reasoner in Protégé. The screenshots shown of inferred individuals are in the repository. **Structure Metrics for the Draft Ontology.** Before mapping, the ontology contained 290 classes, 78 object properties, 18 data properties (96 properties in total), 50 individuals, a maximum hierarchy depth of 2, an average hierarchy breadth of 0.20, attribute richness of 0.25, and relationship density of 0.28. After mapping, it contained 272 classes, 78 object properties, 18 data properties (80 properties in total), 51 individuals, a maximum hierarchy depth of 2, an average hierarchy breadth of 0.21, attribute richness of 0.27, and relationship density of 0.29. Note that, due to the context-window limitations of LLMs, some classes and properties may be removed during the mapping process.

---

[3] https://delaramglp.github.io/airo/, accessed on June 2026.
[4] https://berkeleybop.org/artificial-intelligence-ontology/ , accessed on June 2026.
[5] https://w3c-cg.github.io/dpv/2.3/dpv/ , accessed on June 2026the .
[6] https://delaramglp.github.io/vair/ , accessed on June 2026.
[7] https://artificialintelligenceact.eu/the-act/, accessed on June 15, 2026.
[8] https://github.com/sefeoglu/eu-ai-act-ontology.git , accessed on June 15, 2026